面向软件归档的双层摘要相似试探、风险分流与链上可审计方法

作者姓名*

2026 年 5 月 12 日

摘要

许多软件制品以压缩包、目录或单文件形式在组织间流转。在流水线门禁、外包验收、开源合规检查与内部制品治理场景中，常见需求是：以可控算力完成前置筛查、保留可复核证据，并支持策略按业务动态调优。针对“整包哈希对表层改写不敏感、重型相似检测难以直接用于门禁分流”的工程矛盾，本文提出一套轻量可审计的方法框架。

方法上，本文先对归档路径进行规范化，再构建两层 SHA-256 摘要：第一层用于字节级一致性判定；第二层对文本内容进行表层归一化（去注释、压缩空白、统一换行）后摘要，非文本文件以固定格式引入原始内容摘要。进一步地，本文定义基于第二层摘要哈明距离的分值 $S \in [0, 100]$，并明确其仅用于风险分流，不解释为语义相似度或法律意义上的实质性相似。为增强治理闭环，本文将规则版本、阈值策略、判定分支与证据摘要纳入可追溯日志，并设计链上锚定与离线回放一致性校验流程。

本文同时给出代价敏感阈值优化、消融设计、统计检验与稳健性评估协议，为后续在脱敏数据集上复现实验和形成可发表结果提供标准化模板。该框架定位于“筛查与分流”层，不替代司法鉴定或深度代码相似分析。

关键词：软件制品；归档摘要；表层归一化；相似试探；风险分流；可审计风控

Abstract: Software is often exchanged as archives or folders. CI/CD checks, outsourced acceptance, compliance screening, and internal controls need a lightweight way—with logs and tunable rules—to flag likely duplicates, superficially rewritten bundles, and suspicious cross-submitter drops before legal review. We do *not* issue final infringement judgments.

We normalize paths, then compute two SHA-256 digests. The first hashes raw bytes for strict equality. The second hashes a surface-normalized text concatenation (strip comments, squeeze whitespace, unify newlines); non-text entries contribute fixed-length raw digests as specified. Rules are versioned for reproducibility. A score $S \in [0, 100]$ is derived from bitwise differences between second-layer digests; it is *not* a semantic similarity measure or legal proof, only a triage hint with thresholds. Submitter-aware rules yield actions and audit fields. Empirical numbers will be added after experiments on de-identified data.

Key words: software artifact; archive digest; surface normalization; similarity probe; risk triage; blockchain auditability

*作者单位与电子邮箱

1 引言

1.1 问题背景与核心痛点

实际工程中，压缩包路径写法不统一，且存在刻意“表层改写”规避简单校验的行为。常见问题包括：（1）仅使用整包哈希时，注释或空白修改即可改变摘要，导致疑似同源制品漏检；（2）重型相似检测工具计算开销较高，输出往往面向分析而非门禁处置，审核人员难以快速据此决策；（3）线上治理更需要“自动分流”而非单一相似度排序，即明确哪些样本进入人工复核、哪些按策略直接放行。

1.2 现有做法与本文补充什么

整包哈希适合“字节完全一致”去重，但对“表层改写”不敏感。SimHash 等局部敏感哈希强调“相似输入得到相近指纹”；SHA-256 的扩散特性则相反：微小改动也可能导致摘要显著变化，因此分值大小不能解释为语义接近程度。MOSS、Winnowing 等工具聚焦相似片段检索[5,6]；基于语法树或大模型的方法更偏向深层克隆分析[3,4]，在门禁场景中的成本与时延压力更大[2]。本文面向“压缩包级、规则透明、算力可控”的前置筛查任务，核心目标是缩小人工复核集合并提升审计可追溯性。本文属于工程方法研究，而非通用语义相似理论；输出仅作为筛查证据，不替代司法鉴定或深度检测。

1.3 本文贡献

1. 双层摘要：第一层抓字节级相同；第二层在轻量规整后抓「表面改过仍可能同源」，路径顺序固定，结果可复现。
2. 在两份第二层摘要之间定义分数 S，说明它与「语义相似」不是一回事，只适合配合阈值做分流。
3. 把「是否跨提交者」写进简单规则模板（含 $T_h > T_m$ 与优先级），减轻同一人重复投递带来的误报。
4. 设计链上锚定与离线回放校验流程，使规则版本、判定分支与证据摘要具备不可抵赖的审计属性。
5. 给出从接入到留痕的步骤、评价与消融思路，便于他人按同样协议复现实验。

1.4 研究定位

本文站在「上传网关或流水线门口」：便宜、快、可记账、参数可改。和经典克隆检测论文的差别可以概括为四点：整包（多文件）为对象；输出的是分级与处置建议；阈值与规则要版本号；接口返回足够信息方便复核。

1.5 论文结构

第 2 节对照相近工作；第 3 节写处理步骤、两层摘要、分数 S、规则与复杂度；第 4 节写部署与日志字段；第 5 节写如何做实验与如何读结果；第 6 节小结并谈后续可做事项。

2 相关工作

2.1 软件制品去重

仓库与构建系统大量使用文件哈希或内容寻址存储做去重[1]，厂商门禁也常查恶意与策略。这类办法对「字节一模一样」很稳，但对「改了注释」的包容易当成新药。本文在第一层保留「强一致」摘要，另加第二层对付表面噪声，二者可同时用在门禁前段。

2.2 代码相似检测

文献里既有按文本、记号、树、图等不同粒度比较的综述与工具[3,5,6,8,9]。它们多数回答「有多像、像在哪」，本文回答「这一步该不该进人工队列」，算力以固定长度哈希与比特运算为主，必要时可与重型工具并联。

2.3 局部敏感指纹

SimHash 等让相似文档得到相近指纹，便于设阈值筛「近似重复」[7]。SHA-256 设计目标不同：微小改动也会让输出剧烈变化，因此本文的分数 S 只能绑定在「第二层规整规则之后的那一长串」上理解，不要把它解释成通用的语义相似度。

2.4 工业风控与人机协同

业界共识多是「机器提示、人拍板」[5,10]。制品库与 CI 往往分散提供去重与策略能力；本文把解压、路径整理、两层摘要、打分、规则与写日志串成一条叙述，方便在 API 或流水线里一次性落地。

2.5 区块链存证与可验证审计

已有软件版权治理实践强调证据链完整性与过程可追溯性[11–13]。联盟链可作为“时间戳与证据锚定层”，将判定摘要、策略版本与操作元数据固化，降低链下日志被篡改后的复核不确定性。本文不将链上结果直接等价为侵权结论，而是将其作为筛查过程的可验证佐证。

综上，与相近工作相比：我们把整包多文件当作一个对象；优先输出分流建议而不是相似度榜单；把提交者关系、阈值与规则版本写死可追溯；接口返回命中编号、分数、触发了哪条规则等，便于复查。整体是门口第一道筛子，以后若要更强召回，可再接 SimHash、语法树或模型。

3 方法

3.1 系统流程（五步）

收到一个压缩包（或视作若干路径—内容对）时，按顺序做五件事：

1. 解压或遍历，列出内部文件。
2. 把路径写法统一，便于排序和后面拼接。
3. 路径排好序后，分别算第一层、第二层摘要（见下节）。
4. 用第二层摘要去库里比对，算分数 S；可用阈值 T_d 先缩小候选。
5. 按规则给出风险等级和建议动作，并把关键字段写入日志。

3.2 双层摘要构建

记归档为 $A = \{(p_i, c_i)\}_{i=1}^{n}$：$p_i$ 是路径，c_i 是文件原始字节。所有路径按字典序排好后，依次拼接（顺序写死，任何人复算都得到同一串）。

第一层（规范化归档摘要）H_{can}：对原始内容敏感，用于「字节级是否同一包」。

$$H_{\text{can}} = \text{SHA256}\Big(\bigoplus_{i=1}^{n}\big(\text{norm}(p_i) \parallel \text{Base64}(c_i)\big)\Big), \tag{1}$$

$\text{norm}(\cdot)$ 表示路径规整；$\parallel$ 为拼接；$\bigoplus$ 表示按排序后的次序连接（分隔符与编码在规则版本里写清）。

第二层（表层归一化内容摘要）H_{surf}：文本文件先把正文变成 $\tilde{c}_i$（例如去注释、压空白、统一换行，编码统一到约定字符集）；非文本文件则把「该文件原始字节的 SHA-256」按约定格式嵌进串联串（具体格式同样版本化）。然后：

$$H_{\text{surf}} = \text{SHA256}\Big(\bigoplus_{i=1}^{n}\big(\text{norm}(p_i) \parallel \tilde{c}_i\big)\Big). \tag{2}$$

哪些是文本、哪些是二进制，由部署方按扩展名或文件头等声明；规则一改就要升版本号并留说明，方便事后对账。

3.3 摘要距离分

取两份第二层摘要的十六进制串 h_a, h_b（SHA-256 为 256 比特）。若长度处理有特殊约定（如补齐或罚分），也写在规则版本里。哈明距离 d_H：两串对齐后逐字节异或，数一共有多少个二进制位为 1。

记摘要比特长度为 L（此处 $L = 256$）。$\text{clip}(x, a, b)$ 表示把 x 限制在 $[a, b]$；round 为四舍五入取整。定义：

$$S(h_a, h_b) = \text{clip}\left(\text{round}\left(100 \times \left(1 - \frac{d_H(h_a, h_b)}{L}\right)\right), 0, 100\right). \tag{3}$$

使用时要记住三条：（1）$S = 100$ 表示两份第二层摘要在当前规则下完全一致；（2）$S < 100$

时，分数高低不代表源代码语义有多接近；（3）S 只宜与阈值配合做分流，不能代替侵权或合规裁决。

3.4 风险评估规则（形式化模板）

T_d 用于检索阶段筛候选；T_h、T_m 为处置用阈值，且 $T_h > T_m$。Cross 表示「命中记录来自另一提交者」。可写成：

$$
\begin{aligned}
&(S \geq T_h) \wedge \text{Cross} \Rightarrow \text{高风险，人工复核;} \\
&(S \geq T_h) \wedge \neg\text{Cross} \Rightarrow \text{降级为中风险，人工复核;} \\
&(T_m \leq S < T_h) \Rightarrow \text{中风险，人工复核;} \\
&(S < T_m) \Rightarrow \text{低风险，由策略决定放行或低优先级.}
\end{aligned}
$$

若多条同时适用，优先级为：跨提交者 > 高分段 > 中分段 > 默认低风险。日志里应记下触发了哪一条。

3.5 代价敏感阈值优化

仅以检测指标选择阈值，往往难以反映人工复核成本。为此定义代价敏感目标：在验证集上联合优化漏检、误报与复核工作量。设 FN, FP 分别为漏检与误报样本数，N_{review} 为进入人工复核队列的样本数，$C_{\text{FN}}, C_{\text{FP}}, \lambda$ 为对应代价系数，则

$$J(T_h, T_m) = C_{\text{FN}} \cdot FN + C_{\text{FP}} \cdot FP + \lambda \cdot N_{\text{review}}, \tag{4}$$

$$\min_{T_h, T_m} \; J(T_h, T_m), \quad \text{s.t. } T_h > T_m. \tag{5}$$

工程上可采用网格搜索或贝叶斯优化，在验证集求得候选阈值，再在独立测试集报告最终性能与成本变化。

3.6 算法复杂度与工程意义

若归档总大小为 B，每次只与最近 M 条历史比对，则解压与规整约为 $O(B)$，摘要约 $O(B)$，打分每次固定长度、共 $O(M)$，合计

$$T(B, M) = O(B + M). \tag{6}$$

M 固定时，比对耗时上有顶，便于承诺响应时间。本文只是门禁前的粗筛：大改结构、多语言混编、二进制很多的包都可能仍会漏判或误判，需要更强的工具或人工接手。

4 系统实现

4.1 部署架构

典型链路：接口或流水线接收压缩包 → 解压与路径整理 → 算两层摘要 → 规则引擎（阈值、提交者）→ 返回放行/拦截/进队列，并写数据库或日志。可与现有工单、制品库用消息或回调对接；门禁路径上解析与摘要宜同步完成以满足时延；人工队列可异步写入。

4.2 链上锚定与离线回放

在现有链路上增加证据锚定步骤：对每次判定记录构造证据对象

$$E = (H_{\text{can}}, H_{\text{surf}}, S, \text{risk_level}, \text{rule_id}, \text{rule_ver}, t, \text{operator_id}),$$

并计算

$$e = \text{SHA256}(\text{serialize}(E)).$$

系统将 e、业务索引与时间戳写入联盟链交易；链下保存完整字段。复核时重算 e' 并与链上记录比对，若 $e' = e$ 则可证明该判定记录自上链后未被篡改。该机制用于增强审计可信度，不改变原有分流决策逻辑。

4.3 实现环境与组件映射

结合当前系统实现，前端可采用 Vue，服务端采用 Spring Boot，链下数据库使用 MySQL，文件对象存储使用 MinIO，联盟链网络采用 FISCO BCOS（如 3.11.0 版本）。其中，摘要计算与风险判定在应用服务侧完成，链上仅保存证据锚点与必要索引，避免将源码或敏感内容直接上链。

4.4 审计字段清单

建议至少记下：原始包哈希；两层摘要值；分数 S；风险档位；是否跨提交者；命中哪条规则、规则与阈值版本号；时间；操作者或自动化任务 ID；可选列出命中的历史包编号。

4.5 可解释返回内容

除等级外，接口宜返回：命中谁、分数多少、提交者关系、规则分支、依据哪版策略文档，便于复核同事不必猜。

4.6 规则与阈值版本化

T_d, T_h, T_m、文本规整办法、路径写法等，一律用版本号发布；每条判定绑定版本号，日后审计才能对上号。

5 实验设计与分析框架

5.1 数据集协议

写清数据从哪来、如何标注、训练/验证/测试怎么划分。建议区分：正例（同源包改注释/空白等）；负例（明确不同源）；难负例（技术栈像但业务不同）；跨提交者的高相似例。划分尽量按时间或批次，避免「提前看见未来」。

5.2 基线设置

- A：只对整包或单文件做 SHA-256，不做本文双层摘要。
- B：只用第一层摘要，不做文本规整与第二层串联。
- Ours：两层摘要 + 分数 S + 提交者规则。

5.3 评价指标

- 检测类：Precision、Recall、F1、AUC（按既定标注定义正负）。
- 运营类：复核是否「打得准」、无效复核多少、端到端耗时、队列长度等。
- 系统类：每秒处理多少个包、额外 CPU/内存。

5.4 阈值选择

先在验证集上扫 T_d，尽量不放过该抓的；再结合人力成本选 T_h, T_m（保持 $T_h > T_m$）。

5.5 消融实验（固定四组）

1. 去掉第二层用到的文本规整。
2. 去掉「同提交者降档」规则。
3. 去掉历史窗口限制（改为与全库比）。
4. 系统性地改动阈值，观察指标变化。

5.6 实验承诺

终稿需在脱敏数据集上补齐完整结果，并附数据合规说明、标注口径与复现实验脚本版本信息。

5.7 预期结果与分析框架

对比上，Ours 应比 A 更能收回「只改表面」的同源包；比 B 更能对齐「注释/空白扰动」后的同一归档。分数与阈值一起决定复审边界；不要把 S 当语义排序。

运营上，期望在人力不变时减少无效复核负担、提升高风险样本进入人工队列的比例；端到端延迟需满足业务承诺，可通过历史流量回放与压测联合验证。

阈值放低通常会让更多人进复核、召回相关指标往往变好，但人更忙；阈值抬高则相反。因 $S < 100$ 时分数与语义不同步，阈值应视作运维参数，用验证集加线上回放联合调。

误差典型来源：大面积重写仍会使第二层摘要差异很大；多语言混编；二进制误判类型导致串联失真。

此外要注意样本是否片面、标注口径是否一致、线上环境与试验是否一致。自动化结果只能建议分流；是否违规仍由人与制度判断；涉诉材料组织可参考既有鉴定与技术规范文献[11,14,15]。

5.8 统计检验与稳健性评估

建议对关键指标采用 bootstrap 置信区间报告（例如 95% CI），并在主对比中补充显著性检验（如置换检验或 McNemar 检验，按任务定义选择）。稳健性上，至少报告三类切片结果：按时间切片、按语言类型切片、按提交者关系切片，以评估方法在分布变化下的稳定程度。

5.9 结果呈现模板（终稿填充）

为保证论文可审阅性，建议在终稿至少提供以下三张主表：

- 主对比表：A/B/Ours 的 Precision、Recall、F1、AUC、平均时延、人工复核率；
- 消融表：去规整、去跨提交者规则、去历史窗口后的性能与运营指标变化；
- 审计表：链上写入成功率、上链延迟、离线回放一致性通过率。

若版面允许，可增加一张阈值-成本曲线图，展示在不同 T_h, T_m 下召回与复核负担的权衡关系。

6 结论与展望

针对“整包哈希易漏检表层改写、重型工具不易直接用于门禁分流、治理流程缺少可审计闭环”等问题，本文提出两层摘要、基于摘要距离的分值 S、提交者感知的规则分流，以及链上锚定与回放校验机制。该方法强调低成本、可复现、可追溯和可调优，输出定位为筛查建议而非法律结论。

后续可在同一流水线并联 SimHash、语法特征、许可证扫描或深度克隆检测模型，进一步提升召回能力；在多租户与跨项目场景开展长期 A/B 试验，并探索在线阈值自适应。本文定位为前置轻量筛查层，需与人工审核、组织制度和司法鉴定环节协同运行。

参考文献

[1] DI LUNA G A, et al. The Software Heritage Graph Dataset: Public Software Development Under One Roof[C]//Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering (ICSE). New York: ACM, 2020: 138–148.

[2] FANG C, et al. Large Language Models for Code Analysis: Do LLMs Really Do Their Job?[C]//Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24). Berkeley: USENIX Association, 2024: 5939–5956.

[3] Lee G, Kim J, Choi M S, et al. Review of Code Similarity and Plagiarism Detection Research Studies[J]. Applied Sciences, 2023, 13(20): 11358.

[4] Liu Z, et al. Towards Understanding the Capability of Large Language Models on Code Clone Detection: A Survey[EB/OL]. arXiv:2308.01191, 2023.

[5] Aiken A. Moss: A System for Detecting Software Similarity[EB/OL]. Stanford University. `https://theory.stanford.edu/~aiken/moss/`.

[6] SCHLEIMER S, WILKERSON D S, AIKEN A. Winnowing: Local Algorithms for Document Fingerprinting[C]//Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data. New York: ACM, 2003: 76–85.

[7] HENZINGER M. Finding Near-Duplicate Web Pages: A Large-Scale Evaluation of Algorithms[C]//Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval. New York: ACM, 2006: 284–291.

[8] KAMIYA T, KUSUMOTO S, INOUE K. CCFinder: A Multilinguistic Token-Based Code Clone Detection System for Large Scale Source Code[J]. IEEE Transactions on Software Engineering, 2002, 28(7): 654–670.

[9] JIANG L, MISHERGHI G, SU Z, et al. DECKARD: Scalable and Accurate Tree-Based Detection of Code Clones[C]//Proceedings of the 29th International Conference on Software Engineering. IEEE Computer Society, 2007: 96–105.

[10] NOVAK M, JOY M, KERMEK D. Source-Code Similarity Detection and Detection Tools Used in Academia: A Systematic Review[J]. ACM Transactions on Computing Education, 2019, 19(3): Article 18.

[11] 中华人民共和国司法部. SF/T 0158—2023 软件相似性鉴定技术规范[S]. 北京: 中国标准出版社, 2023.

[12] 国家版权局. 计算机软件著作权登记办法[Z]. 2002.

[13] 国务院. 计算机软件保护条例（2013年修订）[Z]. 2013.

[14] 最高人民法院. 计算机软件著作权侵权案中源代码比对判断相关案例[EB/OL]. 2021.

[15] 最高人民法院知识产权法庭. 网络游戏计算机软件无代码比对时的实质性相似判断[EB/OL]. 2023.